

Introduction

The purpose of this project was to design and deploy a visual object detection system. I decided to have the object of focus be shoes. To accomplish this, I collected my own dataset using the ESP32-S3-EYE camera board by capturing images of four different types of shoes under various conditions. These images were used to train a binary classification model to distinguish between “Shoe” and “NoShoe.”

The trained model was then converted to a TFLite format and deployed on the embedded system. The system processes input images and performs real-time inference directly on the device. The following sections describe the model architecture, data collection process, training methodology, and the performance of the deployed system.

Model Architecture

The neural network architecture was chosen to balance classification performance with the memory and runtime constraints of the ESP32-S3-EYE embedded system. A relatively small convolutional neural network (CNN) was used, consisting of a few convolutional layers followed by pooling and dense layers. This structure was selected because CNNs are effective at extracting spatial features from images while still being efficient enough to run on limited hardware.

The input features consisted of RGB images resized to a smaller resolution (96×128) to reduce computational cost while preserving the original aspect ratio of the captured images. Changing the size of the images also reduced the amount of time that the training took, while making the training reach higher accuracy due to less noise in the images. Initially, the model was training on the full image size of (240 x 320) RGB, which exponentially multiplied the required training time, and also trained on noise in the images at a much higher rate. This led to less clarity in the training as well as less accuracy overall.

Callbacks were also introduced to make the training more efficient. The first callback was Model_Checkpoint, which was used to save the best models according to validation accuracy. The second callback was ReduceLROnPlateau. What ReduceLROnPlateau did was reduce the learning rate after the model had not increased its validation accuracy with a patience of 7 epochs. This led to a higher accuracy because the model would learn as much as it could on a learning rate, and then switch to a lower learning rate to avoid overfitting.

Data and Training

The dataset for this project was collected entirely by myself using the ESP32-S3-EYE camera board. I captured images of four different types of shoes under various conditions, as well as images without shoes to serve as the negative class. No external datasets or shared resources were used. The images were transmitted from the board and saved as JPEG files, then organized into “Shoe” and “NoShoe” categories for training.

To ensure the model could generalize well, I collected data under a variety of conditions. This included different lighting environments, backgrounds (such as a desk, floor, and bed), object positions, distances, and orientations. I also varied how much of the shoe was visible in each image. These variations were important to prevent the model from overfitting to a specific scene or setup and to better reflect real-world usage.

Basic preprocessing was applied by resizing all images to a fixed resolution (96×128) while maintaining the original aspect ratio. Basic augmentation of the images was performed in the code, this led to better generalization. During training, the dataset was split into training, validation, and test sets to evaluate performance at different stages.

The initial model trained on 100 images (50 Shoes, 50 NoShoes). This led to very bad results, so I decided to take nearly 1400 images (700 Shoes, 700 NoShoes). After some more experimentation with the image sizes, the appropriate size of (96×128) was decided upon.

Results

Later in this section are all of the visual results for running my model in both VsCode (Training segment) or in Arduino (Testing segment). In VsCode the best validation accuracy reached 94.74% or nearly 95%. This validation accuracy was reached at epoch 72 out of 126. The precision and recall at that epoch were also very high, reaching 92.27% and 98.09% respectively. The training accuracy at that epoch was 93.87%, which proves that overfitting was not an issue when training on the data for this model.

There was a test_inputs.h file created from the same training file to test if the ESP32-S3-EYE camera board would correctly identify the object. The test_input.h file has two inputs, Input #1 (Shoe), Input #2 (NoShoe). These two classes were chosen to see if the model could detect the difference in the two classes. After running the code in Arduino and connecting to the camera board, good results had come. For Input #1, the board predicted that the object was a Shoe, which was correct, and it did so with a 99.25% confidence. For Input #2, the board predicted that the object was not a Shoe (NoShoe), which was correct, and it did so with a 99.78% confidence.

From the given test set most of the values were correctly predicted, with 77 true positives (TP) and 59 true negatives (TN). There were still some errors with 6 false positives (FP) and 2 false negatives (FN). This was out of a total of 144 available test images. This means that the false positive and false negative rates are very low and are a result of the 95% validation accuracy and not getting the accuracy closer to 100%.

The inference time per input differs slightly, Input #1 had an inference time of 1250903 micro seconds (us) or 1.25 seconds. Input #2 had an inference time of 1250793 (us) or 1.25 seconds. This model is sampling at around 0.8 frames per second (FPS) for each image that has dimensions (96 x 128 x 3).

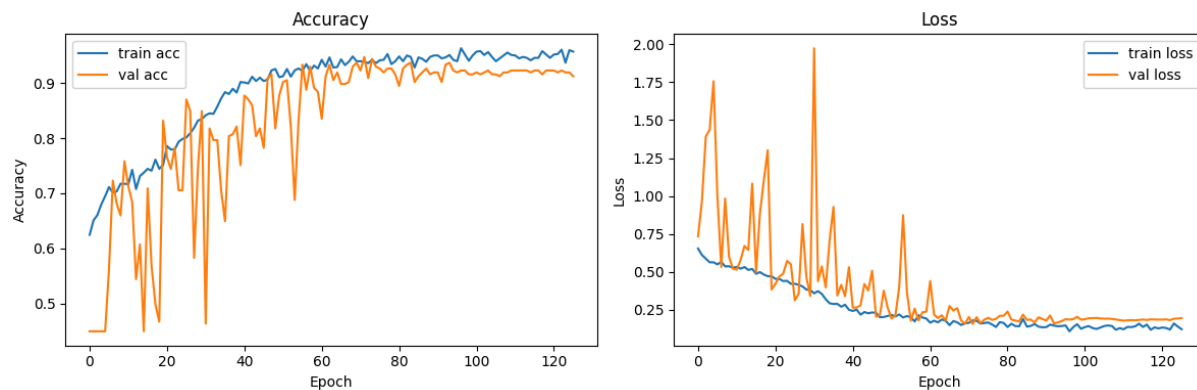
The final accuracies after training for 126 epochs are as follows:

	loss	accuracy	precision	recall	val_loss	val_accuracy	val_precision	val_recall	lr
121	0.128920	0.952715	0.959649	0.946367	0.185926	0.919298	0.952703	0.898089	0.000002
122	0.118663	0.960595	0.950931	0.972318	0.180853	0.922807	0.953020	0.904459	0.000002
123	0.158649	0.936953	0.936207	0.939446	0.189852	0.919298	0.952703	0.898089	0.000002
124	0.138463	0.959720	0.963415	0.956747	0.191303	0.919298	0.952703	0.898089	0.000002
125	0.120485	0.957093	0.958406	0.956747	0.193669	0.912281	0.952055	0.885350	0.000002

However the best validation accuracy received is nearly 95%

```
18/18 [=====] - 1s 22ms/step - loss: 0.1556 - accuracy: 0.9474 - precision: 0.9277 - recall: 0.9809  
[0.15555694699287415, 0.9473684430122375, 0.9277108311653137, 0.9808917045593262]
```

Plots of the training over 126 epochs



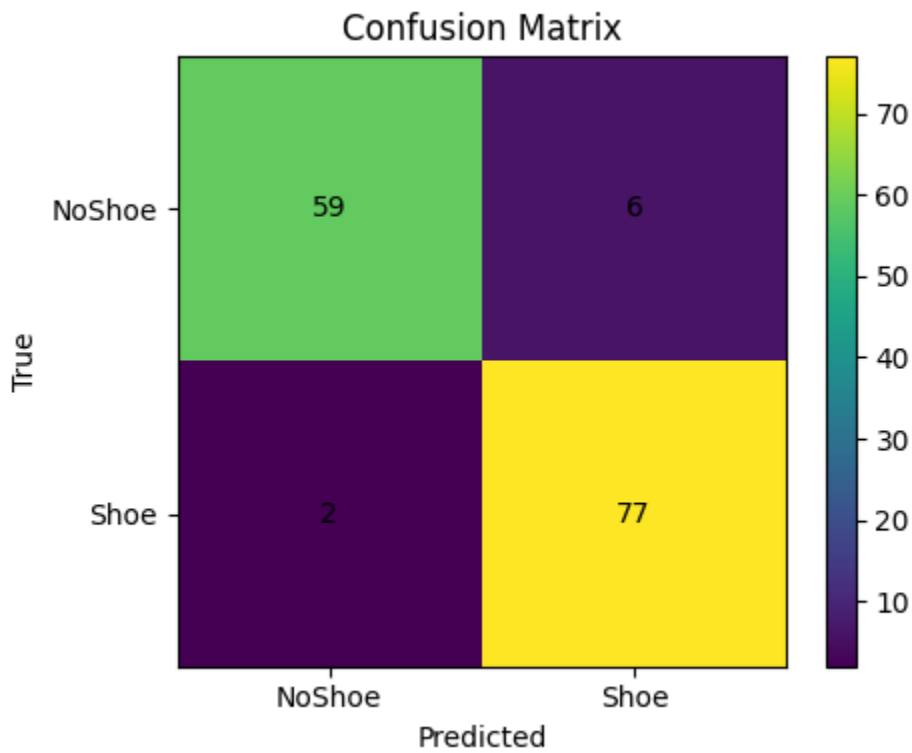
Embedded Results (Via Vscope)

	Real	Keras Predicted	Keras Prob	TFLite Predicted	TFLite Prob	Raw Int8 Output
0	Shoe	Shoe	0.992540	Shoe	0.992188	126
1	NoShoe	NoShoe	0.002205	NoShoe	0.000000	-128

Embedded Results (Via Arduino)

```
----- INPUT 1 -----  
Real: Shoe  
Predicted: Shoe  
Confidence: 0.992187  
Inference time (us): 1250903  
  
----- INPUT 2 -----  
Real: NoShoe  
Predicted: NoShoe  
Confidence: 0.000000  
Inference time (us): 1250793
```

Confusion Matrix (TP + TN + FP + FN)



Summary Table

	Metric	Value
0	Train Accuracy	0.943082
1	Validation Accuracy	0.950355
2	Test Accuracy	0.9375
3	FRR	0.025316
4	FPR	0.092308
5	Input Tensor Shape	(None, 96, 128, 3)
6	Number of Parameters	21201
7	Number of MACs	19470400
8	Sampling Rate (FPS)	0.8 FPS

Discussion

Overall, the model performed well for a binary classification task and it was able to correctly identify “Shoe” vs “NoShoe” in most cases. The system is good, but the images being RGB most likely increased the inference time, and increased the inference speed making it somewhat slow for real-time applications.

The images had to be reduced from the original 240x320x3 size to 96x128x3. The x3 is due to the RGB. Reducing the size of the image reduces the amount of details that the model can train on, but reducing the size also leads to faster training times, and better accuracy.

An important step in training this model was the use of the ReduceLROnPlateau checkpoint. This led to the model not overfitting to the training set, and allowing the validation and testing accuracies to increase to nearly 95%.

Conclusion

In this project, a shoe detection system was successfully developed and deployed on the ESP32-S3-EYE. The model achieved good accuracy and was able to correctly classify images in most cases. This project demonstrated the importance of balancing model performance with embedded system limitations and highlighted the role of dataset quality in achieving reliable results.